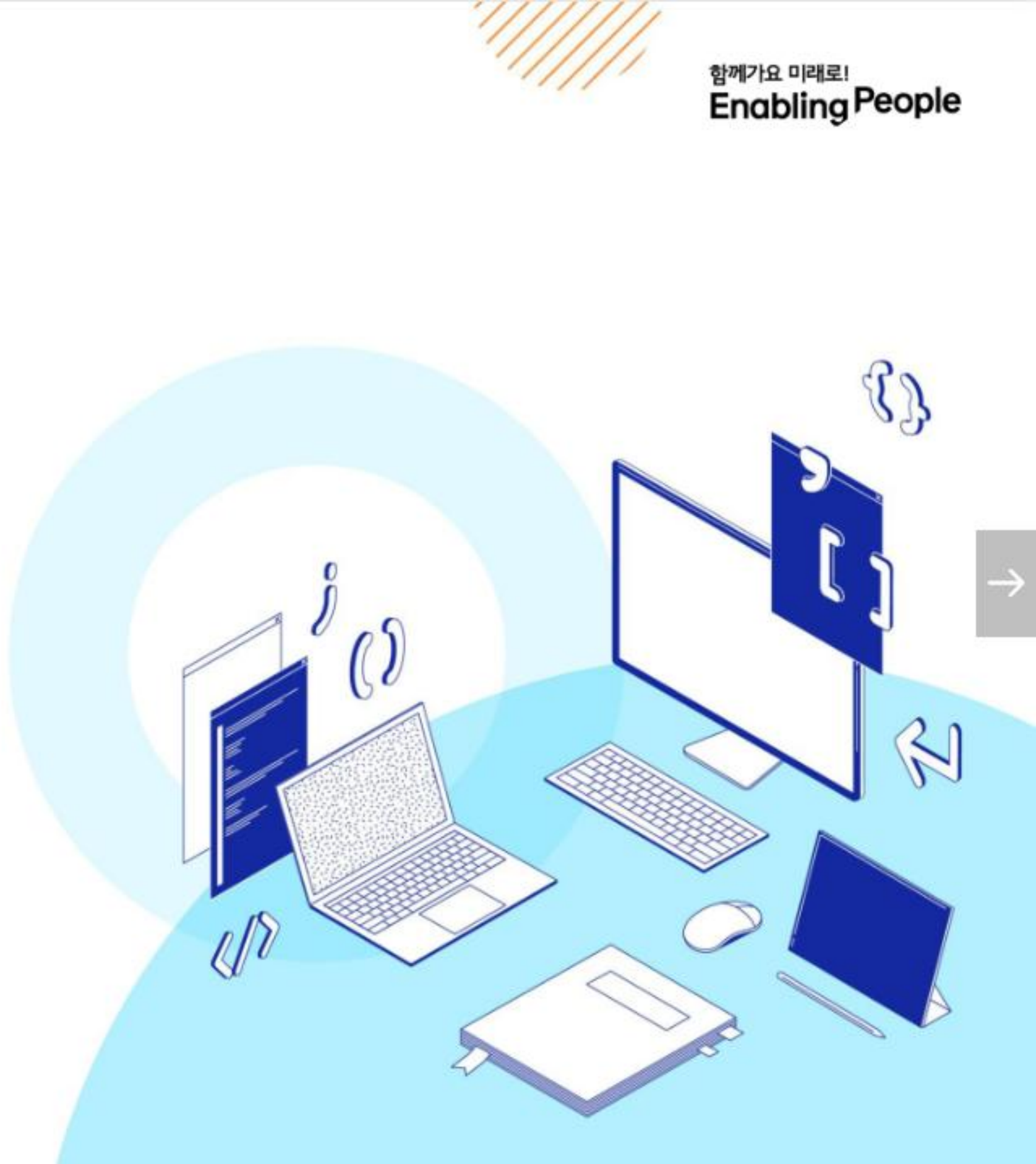


SSAFY

Database

Many To Many Relationships 01

Python



목 차

Many to many relationships

- N:1의 한계
- 중개 모델
- ManyToManyField
- 'through' argument

ManyToManyField

좋아요 기능 구현

- 모델 관계 설정
- 기능 구현

N:1 관계보다 복잡한 관계를 유연하게 설계하는 방안을 고민해봅시다.

- 환자와 의사 간 예약 시스템을 만들 때, N:1 구조만으로는 다양한 관계를 표현하기 어렵습니다.
- 어떻게 하면 다양한 관계에 대한 데이터를 효율적으로 저장할 수 있을까요.

1. Django의 [redacted] 을 사용하면 예약 정보와 부가 정보를 함께 저장할 수 있습니다.
2. [redacted] 를 사용하면 양방향 다대다 관계를 쉽게 구성할 수 있습니다.
3. [redacted] 인자를 활용하면 중간 테이블에 사용자 지정 필드를 추가할 수 있습니다.
4. [redacted], [redacted] 같은 옵션으로 참조 방향성과 충돌을 제어할 수 있습니다.

- 좋아요 기능 등 사용자 행동 기반 관계도 다대다 구조로 표현할 수 있습니다.
- 역참조 설정 시 혼동을 줄이기 위한 설계 전략을 배울 수 있습니다.
- 이러한 기술을 통해 실제 서비스에서 유연하고 확장성 있는 모델링이 가능합니다.

학습 목표

- ✓ N:M 관계가 필요한 상황을 파악하고, Django에서 이를 어떻게 구현하는지 이해한다.
- ✓ ManyToManyField의 기본 동작과 주요 속성들을 설명할 수 있다.
- ✓ 'through'를 활용한 중개 모델 설계법을 습득한다.
- ✓ 좋아요 기능 구현을 통해 관계 설정과 역참조 충돌 해결법을 배운다.
- ✓ Django 모델 설계 시 실무적인 관계 구성 능력을 향상시킨다.

Many to many relationships

다대다 관계 (Many to many relationships)

Many to many relationships

N:M or M:N

한 테이블의 0개 이상의 레코드가
다른 테이블의 0개 이상의 레코드
와 관련된 경우

Many to many (다대다) 관계는 영화와 배우처럼, 한쪽의 여러 개가 다른 쪽의 여러 개와 연결되는 복잡한 관계를 말합니다.

다대다 관계를 설정하면, 한 영화에 여러 배우를 추가하거나, 한 배우가 출연한 모든 영화를 쉽게 찾아낼 수 있습니다.

M:N 관계의 역할과 필요성 이해하기

- '병원 진료 시스템 모델 관계'를 직접 만들어 보기
 - 환자와 의사 2개의 모델을 사용하여 모델 구조 구상
- 기존의 N:1 구조로 환자와 의사 테이블을 구성했을 때 어떤 한계가 있는지 확인
- N:M 관계는 어떻게 데이터베이스에서 관리하는지 확인

➤ 제공된 '99-mtm-practice' 프로젝트를 기반으로 진행

N:1의 한계

의사와 환자 간 모델 관계 설정

- 한 명의 의사에게 여러 환자가 예약할 수 있도록 models 클래스 정의
 - Patient(환자) 모델에 Doctor 모델을 참조하도록 정의
- Migration 까지 진행하여 데이터베이스에 적용

```
# hospitals/models.py

class Doctor(models.Model):
    name = models.TextField()
    def __str__(self):      # print 시 표시될 text 정의
        return f'{self.pk}번 의사 {self.name}'

class Patient(models.Model):
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)
    name = models.TextField()
    def __str__(self):      # print 시 표시될 text 정의
        return f'{self.pk}번 환자 {self.name}'
```

- Django shell 을 활용하여 2명의 의사를 생성

```
$ python manage.py shell -v 2
```

```
In [1]: doctor1 = Doctor.objects.create(name='allie')
```

```
In [2]: doctor1
```

Out[2]: <Doctor: 1번 의사 allie>

```
In [3]: doctor2 = Doctor.objects.create(name='barbie')
```

```
In [4]: doctor2
```

Out[4]: <Doctor: 2번 의사 barbie>

		id INTEGER	* name TEXT
	>	1	allie
	>	2	barbie

〈그림1 DB Many To Many Relationships 01 의사 데이터베이스 확인〉

의사와 환자 데이터 생성 (2/2)

- Django shell 을 활용하여 2명의 환자를 생성하고 환자는 서로 다른 의사에게 예약

```
In [5]: patient1 = Patient.objects.create(name='carol', doctor=doctor1)
```

```
In [6]: patient1
```

```
Out[6]: <Patient: 1번 환자 carol>
```

```
In [7]: patient2 = Patient.objects.create(name='duke', doctor=doctor2)
```

```
In [8]: patient2
```

```
Out[8]: <Patient: 2번 환자 duke>
```

id	name	doctor_
INTEGER	TEXT	bigint
1	carol	1
2	duke	2

<그림2_DB_Many To Many Relationships 01_환자 데이터베이스 확인>

N:1의 한계 상황 (1/3)

- 1번 환자(carol)가 두 의사 모두에게 진료를 받고자 한다면
- 환자 테이블에 1번 환자 데이터가 중복으로 입력될 수 밖에 없음

```
In [9]: patient3 = Patient.objects.create(name='carol', doctor=doctor2)
```

```
In [10]: patient3
```

```
Out[10]: <Patient: 3번 환자 carol>
```

hospitals_doctor

id	name
1	allie
2	barbie

<표1_DB_Many To Many Relationships 01_의사 데이터 테이블>

hospitals_patient

id	name	doctor_id
1	carol	1
2	duke	2
3	carol	2

<표2_DB_Many To Many Relationships 01_환자 테이블에 의사 데이터 추가>

N:1의 한계 상황 (2/3)

- 환자가 진료 받을 의사 정보를 동시에 저장을 시도했을 때는 에러가 발생함

```
In [11]: patient4 = Patient.objects.create(name='carol', doctor=doctor1, doctor2)
Cell In[11], line 1
      patient4 = Patient.objects.create(name='carol', doctor=doctor1, doctor2)
                                             ^
SyntaxError: positional argument follows keyword argument
```

hospitals_doctor

id	name
1	allie
2	barbie

<표1_DB_Many To Many Relationships 01_의사 데이터 테이블>

hospitals_patient

id	name	doctor_id
1	carol	1
2	duke	2
3	carol	2
4	carol	1, 2

<표3_DB_Many To Many Relationships 01_환자 테이블에 의사 데이터 중복>

N:1의 한계 상황 (3/3)

- 동일한 환자지만 다른 의사에게도 진료 받기 위해 새로운 예약 데이터를 만들어야 하며 이 때, 동일한 환자 정보를 또 작성하여 저장됨
 - 데이터의 중복이 발생하면 나중에 환자의 정보가 바뀌었을 때, 모든 예약 정보를 하나하나 찾아서 고쳐야 하는 문제가 발생하고, 실수로 하나라도 누락한다면 데이터의 일관성이 깨지게 됨
- 외래 키 컬럼에 '1, 2' 형태로 저장하는 것은 DB 타입 문제로 불가능
 - 제1정규형을 만족하지 못하기 때문에 사용이 불가함
- 의사의 정보도 외래 키로 참조하고, 환자의 정보도 외래 키로 참조하는 별개의 테이블을 만든다면 위에 언급된 문제를 해결할 수 있게 됨
 - 즉, “예약 테이블을 따로 만들자”



제 1정규형
테이블의 모든 칸에 더 이상 쪼갤 수 없는 하나의 값만 넣어야 함

<개념1_DB_Many To Many Relationships 01_제1정규형>



중개 모델

중개 모델

중개 모델

다대다 관계에서 두 모델을 연결하는 역할을 하는, 특별한 기능을 가진 모델

의사와 환자가 '예약'이라는 관계로 연결될 때, 단순히 '의사 A와 환자 B가 연결됐다'는 사실 외에 '언제 예약했는지', '예약의 상태는 무엇인지' 같은 정보를 함께 저장하는 모델입니다

중개 모델을 사용하면 Doctor 모델이나 Patient 모델에 없는, 예약 행위 자체에 대한 상세한 정보를 담을 수 있어 복잡한 현실 세계의 관계를 더욱 정교하게 표현할 수 있습니다.

TIP

- 중개 - 직접 당사자가 되지 않고, 거래나 계약이 이루어지도록 돕는 역할 (공인 중개사)
- 중계 - 중간에서 이어받아 다른 곳으로 전달하거나 연결하는 것 (중계 방송)

예약 모델 생성

- 환자 모델의 외래 키를 삭제하고 별도의 예약 모델을 새로 생성
- 예약 모델은 의사와 환자에 각각 N:1 관계를 가짐

```
# hospitals/models.py
# 외래 키 삭제
class Patient(models.Model):
    name = models.TextField()
    def __str__(self):
        return f'{self.pk}번 환자 {self.name}'

# 중개모델 작성
class Reservation(models.Model):
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE)
    def __str__(self):
        return f'{self.doctor_id}번 의사의 {self.patient_id}번 환자'
```

hospitals_reservation	
Columns	
id	INTEGER
doctor_id	bigint
patient_id	bigint

<그림3_DB_Many To Many Relationships 01_예약 테이블 구조 확인>

예약 데이터 생성

- 데이터베이스 초기화 후 Migration 진행 및 shell 실행
- 의사와 환자 데이터 생성 후 예약 정보 추가하기

```
In [1]: doctor1 = Doctor.objects.create(name='allie')
```

```
In [2]: doctor1
```

```
Out[2]: <Doctor: 1번 의사 allie>
```

```
In [3]: patient1 = Patient.objects.create(name='carol')
```

```
In [4]: patient1
```

```
Out[4]: <Patient: 1번 환자 carol>
```

```
In [5]: reservation1 = Reservation.objects.create(doctor=doctor1, patient=patient1)
```

```
In [6]: reservation1
```

```
Out[6]: <Reservation: 1번 의사의 1번 환자>
```

예약 정보 조회

- 의사와 환자가 예약 모델을 통해 각각 본인의 진료 내역 확인
 - 각각의 의사 혹은 환자 데이터에서 중개 테이블의 역참조를 통해 예약 내역을 확인할 수 있음

의사 -> 예약 정보 찾기

```
In [7]: doctor1.reservation_set.all()
```

```
Out[7]: <QuerySet [<Reservation: 1번 의사의 1번 환자>]>
```

환자 -> 예약 정보 찾기

```
In [8]: patient1.reservation_set.all()
```

```
Out[8]: <QuerySet [<Reservation: 1번 의사의 1번 환자>]>
```


추가 예약 생성

- 1번 의사에게 새로운 환자 예약 생성

```
In [9]: patient2 = Patient.objects.create(name='duke')
```

```
In [10]: patient2
```

```
Out[10]: <Patient: 2번 환자 duke>
```

```
In [11]: reservation2 = Reservation.objects.create(doctor=doctor1, patient=patient2)
```

```
In [12]: reservation2
```

```
Out[12]: <Reservation: 1번 의사의 2번 환자>
```

hospitals_reservation			id	doctor_id	patient_id
Columns			INTEGER	bigint	bigint
	id		INTEGER		
	doctor_id		bigint		
	patient_id		bigint		
		>	1	1	1
		>	2	1	2

<그림4_DB_Many To Many Relationships 01_예약 데이터베이스 데이터 확인>

예약 정보 조회

- 1번 의사의 전체 예약 정보 조회

- 예약 테이블의 역참조를 활용해서 어떤 예약이 있는지 확인할 수 있음

```
# 의사 -> 환자 목록
```

```
In [13]: doctor1.reservation_set.all()
```

```
Out[13]: <QuerySet [<Reservation: 1번 의사의 1번 환자>, <Reservation: 1번 의사의 2번 환자>]>
```

- 이렇게 다대다 관계(Many to many relationship)의 경우 중개 테이블을 생성해서 데이터를 효율적으로 관리할 수 있음
- Django에서는 'ManyToManyField'를 통해 두 모델 간의 중간 테이블(중개 모델)이 자동으로 만들어짐

ManyToManyField

ManyToManyField

ManyToManyField()

 | M:N 관계 설정 모델 필드

이 필드를 설정하면 Django는 자동으로 중간 테이블(중개 모델)을 생성하여 각 모델 간의 관계를 관리합니다.

모델 클래스 내부에 필드로 정의하며, 어느 모델에 정의해도 관계는 동일하게 유지됩니다.

ManyToManyField의 필드명은 다대다 관계를 나타내기 위해 복수형으로 작성을 권장합니다.

ManyToManyField의 필수 인자는 관계를 가지는 모델 클래스를 작성합니다.

ManyToManyField 조작

- .add() 메서드

- 중개 테이블에 새로운 데이터를 추가할 때 사용
- 인자로 연결할 대상 모델의 인스턴스를 넣어서 사용

```
# 단일 인스턴스 추가
patient.doctors.add(doctor1)
# 여러 인스턴스 한 번에 추가
patient.doctors.add(doctor2, doctor3)
```

- .remove() 메서드

- 중개 테이블에 있는 데이터를 삭제할 때 사용
- 인자로 전달한 인스턴스를 중개 테이블에서 제거하며, 대상 객체 자체는 삭제되지 않음

```
# 단일 관계 삭제
patient.doctors.remove(doctor1)
# 여러 관계 한 번에 삭제
patient.doctors.remove(doctor2, doctor3)
```

※ 자세한 사용 방법은 다음 장에 진행되는 실습에서 확인

Django ManyToManyField 실습 (1/6)

- 환자 모델에 ManyToManyField 작성
 - 의사 모델에 작성해도 상관 없으며 참조/역참조 관계만 잘 기억할 것
 - ※ 환자 모델은 의사 모델을 참조하며, 의사 모델은 역참조를 통해 환자 모델에 접근

```
# hospitals/models.py

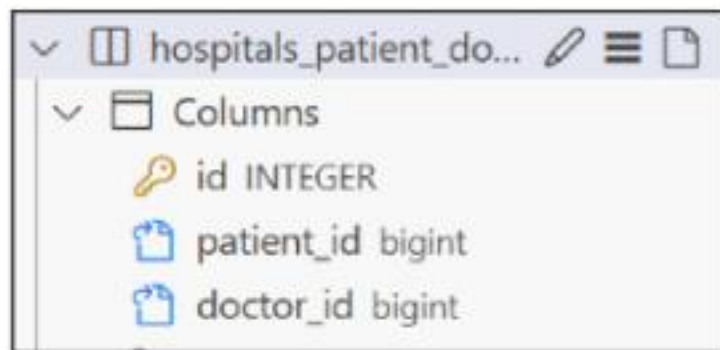
class Patient(models.Model):
    # ManyToManyField 작성
    doctors = models.ManyToManyField(Doctor)
    name = models.TextField()

    def __str__(self):
        return f'{self.pk}번 환자 {self.name}'

# Reservation Class 주석 처리
```


Django ManyToManyField 실습 (2/6)

- 데이터베이스 초기화 후 Migration 진행 및 shell 실행
- 생성된 중개 테이블 hospitals_patient_doctors 확인
 - 중개 테이블 이름은 '앱이름_모델클래스명_필드명' 으로 생성됨



hospitals_patient_do...	
Columns	
id	INTEGER
patient_id	bigint
doctor_id	bigint

<그림5_DB_Many To Many Relationships 01_중개 테이블 구조 확인>

※ 필드가 아닌 테이블이 생성됨을 유의

- 의사 1명과 환자 2명에 대한 데이터 생성

```
In [1]: doctor1 = Doctor.objects.create(name='allie')  
  
In [2]: patient1 = Patient.objects.create(name='carol')  
  
In [3]: patient2 = Patient.objects.create(name='duke')
```

Django ManyToManyField 실습 (3/6)

- 환자1의 예약에 의사1을 추가하고 예약 내역 조회하기
 - 환자의 경우 클래스 내부에 필드를 가지고 있어 참조로 조회
 - 의사의 경우 역참조를 통해 환자 데이터 조회

```
# patient1이 doctor1에게 예약
In [4]: patient1.doctors.add(doctor1)

# patient1 - 자신이 예약한 의사목록 확인
In [5]: patient1.doctors.all()
Out[5]: <QuerySet [<Doctor: 1번 의사 allie>]>

# doctor1 - 자신의 예약된 환자목록 확인
In [6]: doctor1.patient_set.all()
Out[6]: <QuerySet [<Patient: 1번 환자 carol>]>
```

Django ManyToManyField 실습 (4/6)

- 의사1이 환자2에 대한 예약 추가하고 예약 내역 조회하기
 - 의사는 역참조 매니저를 통해 환자를 추가할 수 있음

```
# doctor1이 patient2을 예약
```

```
In [7]: doctor1.patient_set.add(patient2)
```

```
# doctor1 - 자신의 예약 환자목록 확인
```

```
In [8]: doctor1.patient_set.all()
```

```
Out[8]: <QuerySet [<Patient: 1번 환자 carol>, <Patient: 2번 환자 duke>]>
```

```
# patient1, 2 - 자신이 예약한 의사목록 확인
```

```
In [9]: patient1.doctors.all()
```

```
Out[9]: <QuerySet [<Doctor: 1번 의사 allie>]>
```

```
In [10]: patient2.doctors.all()
```

```
Out[10]: <QuerySet [<Doctor: 1번 의사 allie>]>
```


Django ManyToManyField 실습 (5/6)

- 중개 테이블 (hospitals_patient_doctors)에서 예약 현황 확인
 - 환자와 의사 정보가 외래키로 저장되어 있음을 확인
 - 데이터의 중복 없이 정보를 효율적으로 저장하고 있음



hospitals_patient_doctors	id	patient_id	doctor_id
Columns	id INTEGER	patient_id bigint	doctor_id bigint
	> 1	1	1
	> 2	2	1

〈그림6_DB_Many To Many Relationships 01_중개 테이블에 데이터 저장 확인〉

Django ManyToManyField 실습 (6/6)

- 예약 취소하기 (삭제)
- 이전에는 Reservation을 찾아서 지워야 했다면, 이제는 .remove() 로 삭제 가능

patient2가 doctor1 진료 예약 취소

In [11]: patient2.doctors.remove(doctor1)

In [12]: patient2.doctors.all()

Out[12]: <QuerySet []>

In [13]: doctor1.patient_set.all()

Out[13]: <QuerySet [<Patient: 1번 환자 carol>]>

doctor1이 patient1 진료 예약 취소

In [14]: doctor1.patient_set.remove(patient1)

In [15]: doctor1.patient_set.all()

Out[15]: <QuerySet []>

In [16]: patient2.doctors.all()

Out[16]: <QuerySet []>

기본 ManyToManyField의 한계

- 기본 ManyToManyField 로 생성된 예약 중개 테이블은 의사와 환자의 외래 키 정보만 저장하고 있음
- 만약 예약 중개 테이블에 병의 증상, 예약 일정, 방문 횟수에 대한 추가 정보가 필요한 경우, 기본 ManyToManyField를 그대로 사용할 수 없음
- 추가 정보를 저장하기 위해서는 사용자가 직접 중개 테이블을 정의해야 함
 - 사용자가 직접 중개 테이블을 정의하면 .add(), .remove()에 메서드를 사용할 수 없음
- ManyToManyField에 'through' 속성을 통해 사용자가 작성한 중개 테이블을 등록하면 추가 정보 저장 및 .add(), .remove() 메서드를 그대로 활용할 수 있게 됨

'through' argument

ManyToManyField의 'through' 속성

'through' 속성

중개 테이블에 '추가 데이터'를 사용해
M:N 관계를 형성하려는 경우에 사용

'through' 속성 실습 (1/6)

- Reservation Class 재작성 및 through 설정
 - 이제는 예약 정보에 "증상(symptom)"과 "예약일(reserved_at)"이라는 추가 필드가 생김

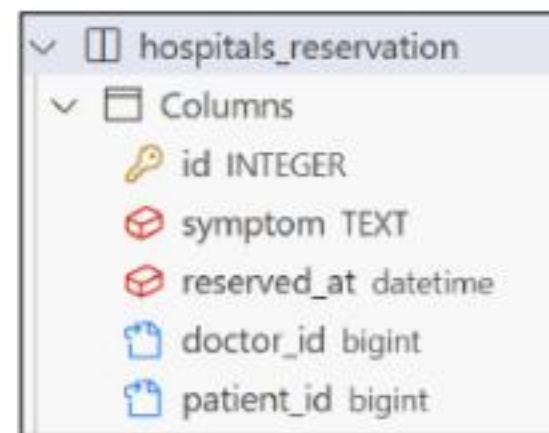
```
class Patient(models.Model):
    doctors = models.ManyToManyField(Doctor, through='Reservation')
    name = models.TextField()
    def __str__(self):
        return f'{self.pk}번 환자 {self.name}'

class Reservation(models.Model):
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE)
    symptom = models.TextField()
    reserved_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f'{self.doctor.pk}번 의사의 {self.patient.pk}번 환자'
```


'through' 속성 실습 (2/6)

- 데이터베이스 초기화 후 Migration 진행 및 shell 실행
- 의사 1명과 환자 2명 생성

```
In [1]: doctor1 = Doctor.objects.create(name='allie')  
  
In [2]: patient1 = Patient.objects.create(name='carol')  
  
In [3]: patient2 = Patient.objects.create(name='duke')
```



hospitals_reservation	
Columns	
id	INTEGER
symptom	TEXT
reserved_at	datetime
doctor_id	bigint
patient_id	bigint

<그림7_DB_Many To Many
Relationships 01_직접 구현한 중개 테이블
구조 확인>

'through' 속성 실습 (3/6)

- 예약 생성 방법 - 1
 - Reservation class를 통한 예약 생성

```
In [4]: reservation1 = Reservation(doctor=doctor1, patient=patient1, symptom='headache')
```

```
In [5]: reservation1.save()
```

```
In [6]: doctor1.patient_set.all()
```

```
Out[6]: <QuerySet [<Patient: 1번 환자 carol>]>
```

```
In [7]: patient1.doctors.all()
```

```
Out[7]: <QuerySet [<Doctor: 1번 의사 allie>]>
```

'through' 속성 실습 (4/6)

• 예약 생성 방법 - 2

- Patient 또는 Doctor의 인스턴스를 통한 예약 생성 (`through_defaults`)
- `.add()` 메서드의 `through_defaults` 인자에 {필드명: 값} 형태로 전달하면 중개 모델의 해당 필드에 값이 저장됨

```
In [8]: patient2.doctors.add(doctor1, through_defaults={'symptom': 'flu'})
```

```
In [9]: doctor1.patient_set.all()
```

```
Out[9]: <QuerySet [<Patient: 1번 환자 carol>, <Patient: 2번 환자 duke>]>
```

```
In [10]: patient2.doctors.all()
```

```
Out[10]: <QuerySet [<Doctor: 1번 의사 allie>]>
```


'through' 속성 실습 (5/6)

- 중개 테이블 (hospitals_reservation)에 생성된 예약 정보 확인
 - 두 방법 모두 중개 테이블에 잘 저장되었음을 확인

hospitals_reservation			id INTEGER	* symptom TEXT	* reserved_at datetime	* doctor_id bigint	* patient_id bigint
Columns							
id INTEGER							
symptom TEXT							
reserved_at datetime							
doctor_id bigint							
patient_id bigint							
		>	1	headache	2023-01-01 12:00:00	1	1
		>	2	flu	2023-01-01 13:00:00	1	2

<그림8_DB_Many To Many Relationships 01_직접 구현한 중개 테이블 데이터 확인>

'through' 속성 실습 (6/6)

- 생성과 마찬가지로 의사와 환자 모두 각각 예약 삭제 가능
- 직접 중개 테이블의 인스턴스를 활용한 삭제

```
# Reservation 을 이용한 예약 삭제  
In [11]: reservation1.delete()  
Out[11]: (1, {'hospitals.Reservation': 1})
```

※ 중개 테이블을 통한 삭제는 delete 메서드 사용

- 의사 데이터를 통해 환자와의 예약 삭제

```
# 의사를 통한 예약 삭제  
In [12]: doctor1.patient_set.remove(patient2)
```

※ 환자 데이터를 통해서도 동일하게 예약 삭제 가능

M:N 관계 주요 사항 정리

- M:N 관계로 맺어진 두 테이블에는 물리적인 변화가 없음
- ManyToManyField는 중개 테이블을 자동으로 생성
- ManyToManyField는 M:N 관계를 맺는 두 모델 어디에 위치해도 상관 없음
 - 대신 필드 작성 위치에 따라 참조와 역참조 방향을 주의할 것
- N:1은 완전한 종속의 관계였지만 M:N은 종속적인 관계가 아니며 '의사에게 진찰받는 환자' & '환자를 진찰하는 의사' 이렇게 2가지 형태 모두 표현 가능

ManyToManyField

ManyToManyField 특징

ManyToManyField(to, **options)

M:N 관계 설정 시 사용하는 모델 필드

어느 모델에서든 관련 객체에 접근할 수 있는 **양방향** 관계
동일한 관계는 한 번만 저장되며 **중복되지 않음**

ManyToManyField의 대표 인자 3가지

1. `related_name`
 - 역참조 이름을 변경할 때 설정하는 인자
2. `symmetrical`
 - 관계 설정 시 대칭에 대한 설정을 하는 인자
3. `through`
 - 직접 생성한 중개 테이블을 등록하는 인자

'related_name' argument

- 역참조시 사용하는 manager name을 변경
 - 기본 값인 '역참조모델명_set'을 다른 이름으로 변경할 때 사용
 - 이름을 변경하면 더 이상 기본 값('역참조모델명_set')을 사용할 수 없음

```
class Patient(models.Model):  
    doctors = models.ManyToManyField(Doctor, related_name='patients')  
    name = models.TextField()
```

```
# 변경 전  
doctor.patient_set.all()
```

```
# 변경 후 (변경 후 이전 manager name은 사용 불가)  
doctor.patients.all()
```

'symmetrical' argument (1/2)

- 관계 설정 시 대칭 유무 설정
- ManyToManyField가 동일한 모델을 가리키는 정의에서만 사용
- 기본 값 : True

예시

```
class Person(models.Model):  
    friends = models.ManyToManyField('self')  
    # friends = models.ManyToManyField('self', symmetrical=False)
```

'symmetrical' argument (2/2)

- source 모델: 관계를 시작하는 모델
- target 모델: 관계의 대상이 되는 모델
- symmetrical 값이 True인 경우
 - source 모델의 인스턴스가 target 모델의 인스턴스를 참조하면 자동으로 target 모델 인스턴스도 source 모델 인스턴스를 자동으로 참조하도록 함(대칭)
 - 즉, 내가 당신의 친구라면 자동으로 당신도 내 친구가 됨
- symmetrical 값이 False인 경우
 - True와 반대 (대칭되지 않음)

'through' argument

- 사용하고자 하는 중개모델을 지정
- 일반적으로 "추가 데이터를 M:N 관계와 연결하려는 경우"에 활용

```
class Patient(models.Model):  
    doctors = models.ManyToManyField(Doctor, through='Reservation')  
    name = models.TextField()  
  
class Reservation(models.Model):  
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)  
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE)  
    symptom = models.TextField()  
    reserved_at = models.DateTimeField(auto_now_add=True)
```

좋아요 기능 구현

모델 관계 설정

다대다 관계 설정

Many to many relationships

한 테이블의 0개 이상의 레코드가
다른 테이블의 0개 이상의 레코드와 관련된 경우
❖ 양쪽 모두에서 N:1 관계를 가짐

좋아요 기능의 모델 관계 설정 (1/2)

- Article (M) - User (N)
 - 게시글은 좋아요가 없을 수도 있고, 여러 개 존재할 수 있음
 - 사용자도 게시글에 좋아요를 한 번도 누르지 않았을 수도 있고, 여러 개의 게시글에 좋아요를 누를 수 있음
- Article 클래스에 ManyToManyField 작성
 - 필드명은 직관적이고 복수형으로 작성

```
# articles/models.py
class Article(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    like_users = models.ManyToManyField(settings.AUTH_USER_MODEL)
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```


좋아요 기능의 모델 관계 설정 (2/2)

- Migration 진행 시 에러 발생 확인
 - 유저가 게시글을 역참조 할 때 역참조 이름에 대한 충돌이 발생하고 있음

```
$ python manage.py makemigrations
SystemCheckError: System check identified some issues:

ERRORS:
articles.Article.like_users: (fields.E304) Reverse accessor 'User.article_set' for 'articles.Article.like_users' clashes with reverse accessor for 'articles.Article.user'.
        HINT: Add or change a related_name argument to the definition for 'articles.Article.like_users' or 'articles.Article.user'.
articles.Article.user: (fields.E304) Reverse accessor 'User.article_set' for 'articles.Article.user' clashes with reverse accessor for 'articles.Article.like_users'.
        HINT: Add or change a related_name argument to the definition for 'articles.Article.user' or 'articles.Article.like_users'.
```

<그림9_DB_Many To Many Relationships 01_역참조 이름 충돌 에러>

역참조 매니저 충돌 (1/2)

- 게시글의 작성자 저장 위해 설정한 ForeignKey 의 역참조
 - "유저가 작성한 게시글"을 확인하기 위해서는 'article_set' 를 사용해야 함
 - `user.article_set.all()`
- 좋아요 기능을 위해 설정한 ManyToManyField의 역참조
 - "유저가 좋아요 한 게시글"을 확인하기 위해서는 'article_set'을 사용해야 함
 - `user.article_set.all()`
- 'article_set' 의 역참조 이름이 겹치게 되어 충돌이 발생하고 있음

역참조 매니저 충돌 (2/2)

- User와 Article 모델 사이에 ForeignKey(작성자)와 ManyToManyField(좋아요) 두 관계가 공존하면서 'article_set'이라는 역참조 이름이 충돌하여 어떤 데이터에 접근해야 하는지 알 수 없게 됨
- 'user가 작성한 글 (user.article_set)'과 'user가 좋아요를 누른 글(user.article_set)'을 구분할 수 있도록 코드 수정이 필요함
- 두 관계 중 하나에 related_name을 설정해야 하는데, 일반적으로 ManyToManyField쪽에 related_name을 추가하는 것을 권장

M:N 관계를 바꾸는 것이 더 좋은 이유 2가지

1. '소유' 관계의 기본값 유지

- ForeignKey(1:N) 관계는 보통 '소유'나 '소속'의 의미를 가짐 (예: 게시글의 소유자는 유저)
- Django의 기본 역참조 이름인 'article_set'은 'User가 소유한 Article의 집합'이라는 매우 직관적인 의미로 해석됨
- 이처럼 가장 중요하고 대표적인 관계의 기본값을 그대로 유지하는 것이 코드의 예측 가능성을 높임

2. '행위' 관계의 명시적 표현

- ManyToManyField(M:N) 관계는 보통 '참여' 나 '행위'의 의미를 가집니다. (예: 유저가 게시글에 '좋아요를 누르는' 행위)
- 이 관계에 `related_name='like_articles'`처럼 행위의 의미가 드러나는 구체적인 이름을 붙여주면, 코드의 가독성이 크게 향상됨

충돌 해결하기 (1/2)

- `user.article_set.all()`
 - "이 유저가 작성한 모든 글" (소유 관계, 기본)
 - `user.like_articles.all()`
 - "이 유저가 좋아요를 누른 모든 글" (행위 관계, 명시적)
- 핵심적인 '소유' 관계(N:1)는 기본값을 유지하고, 부가적인 '행위' 관계(M:N)에 구체적인 이름을 붙여주는 것으로 설계하기

충돌 해결하기 (2/2)

- related_name 작성 후 Migration 재진행

```
# articles/models.py
class Article(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    like_users = models.ManyToManyField(settings.AUTH_USER_MODEL, related_name='like_articles')
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

- 생성된 중개 테이블 확인



〈그림10_DB_Many To Many Relationships 01_중개 테이블 확인〉

User - Article간 사용 가능한 전체 related manager

- article.user
 - 게시글을 작성한 유저 - N:1
- user.article_set
 - 유저가 작성한 게시글(역참조) - N:1
- article.like_users
 - 게시글을 좋아요 한 유저 - M:N
- user.like_articles
 - 유저가 좋아요 한 게시글(역참조) - M:N

기능 구현

좋아요 기능 구현 (1/4)

- 좋아요 기능을 담당할 URL 등록
 - 어떤 게시글에 좋아요가 눌렸는지 정보를 전달하기 위해 '게시글 pk' 정보를 variable routing으로 추가

```
# articles/urls.py

app_name = 'articles'
urlpatterns = [
    path('', views.index, name='index'),
    path('<int:pk>/', views.detail, name='detail'),

    ... (중략)

    path('<int:article_pk>/likes/', views.likes, name='likes'),
]
```

좋아요 기능 구현 (2/4)

- view 함수 구현

- 사용자가 게시글의 좋아요 목록에 있으면 좋아요 취소 (사용자 정보 삭제)
- 사용자가 게시글의 좋아요 목록에 없으면 좋아요 추가 (사용자 정보 추가)

```
# articles/views.py

@login_required
def likes(request, article_pk):
    article = Article.objects.get(pk=article_pk)

    if request.user in article.like_users.all():
        article.like_users.remove(request.user)
    else:
        article.like_users.add(request.user)

    return redirect('articles:index')
```

좋아요 기능 구현 (3/4)

- index 템플릿에서 각 게시 글에 좋아요 버튼 출력
 - 좋아요 버튼의 value도 현재 상태에 따라 다르게 출력하도록 코드 작성

```
<!-- articles/index.html -->
{% for article in articles %}
... (중략)
<form action="{% url 'articles:likes' article.pk %}" method="POST">
  {% csrf_token %}
  {% if request.user in article.like_users.all %}
    <input type="submit" value="좋아요 취소">
  {% else %}
    <input type="submit" value="좋아요">
  {% endif %}
</form>
<hr>
{% endfor %}
```


좋아요 기능 구현 (4/4)

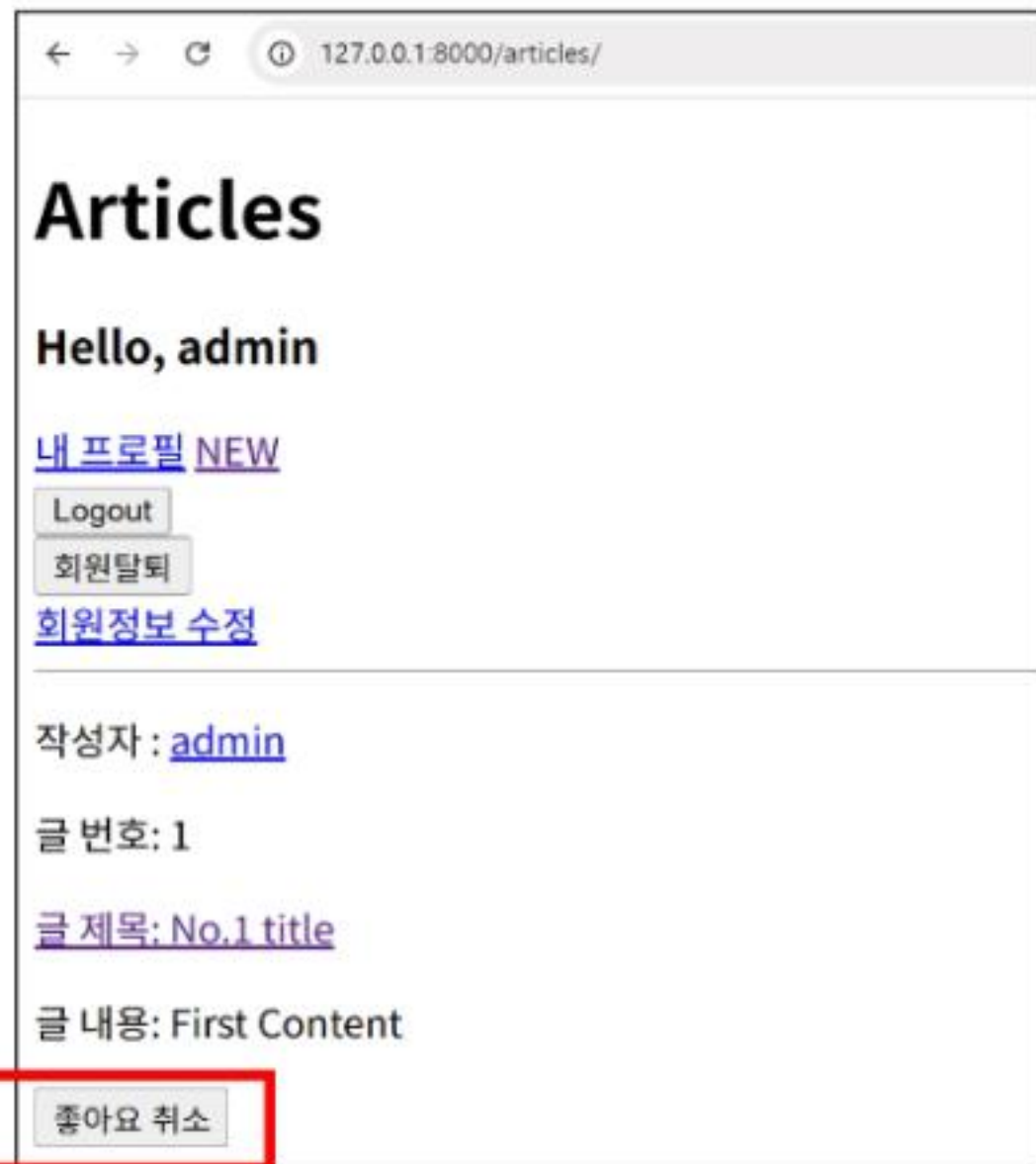
- 좋아요 버튼 출력 및 동작 확인
 - 좋아요 버튼을 누를 때 마다 버튼의 텍스트가 변경되는지 확인



- 좋아요 버튼 클릭 후 테이블 확인

articles_article_like_users Columns id INTEGER article_id bigint user_id bigint		id INTEGER 1	article_id bigint 1	user_id bigint 1
---	--	--------------------	---------------------------	------------------------

<그림12_DB_Many To Many Relationships 01_좋아요 테이블에 데이터 저장 확인>



<그림11_DB_Many To Many Relationships 01_좋아요 버튼 동작 확인>

Many to many relationships

- 2919. M:N 복습 - 모델 작성
- 3061. diary M:N 관계 추가
- 1910. 온라인 쇼핑몰 시스템 - M:N 모델 정의
- 1911. 온라인 쇼핑몰 시스템 - M:N 모델 개선
- 1912. 이벤트 참여자 목록 관리 서비스 - M:N 모델 정의
- 1913. 이벤트 참여자 목록 관리 서비스 - 중개 모델
- 1914. 이벤트 참여자 목록 관리 서비스 - 모델 수정

좋아요 기능 구현

- 1974. M:N 복습 - 좋아요 기능 구현
- 3063. 추천 기능 구현
- 3062. diary M:N 기능 구현

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 N:1 관계에서 발생할 수 있는 문제는?

- a) 데이터 중복 발생
- b) 외래키가 두 개 필요함
- c) 하나의 테이블만 생성됨
- d) shell 명령어를 사용할 수 없음

3 중개 테이블에 추가 정보를 저장하기 위해 사용하는 ManyToManyField의 속성 이름은?

- a) related_name
- b) symmetrical
- c) through
- d) default

2 다음 중 Django에서 다대다 관계를 설정할 때 사용하는 필드는?

- a) OneToOneField
- b) ForeignKey
- c) ManyToManyField
- d) CharField

4 다음 문장이 올바른지 O, X로 답하시오.

중개 모델은 두 객체 간 관계 외에 추가 정보를 함께 저장할 수 있다.

Q 각 문제에 올바른 답안을 생각해봅시다

5 다음 중 ManyToManyField에 연결된 데이터를 삭제하는 메서드는?

- a) .clear()
- b) .add()
- c) .remove()
- d) .delete()

7 다음 중 symmetrical=True의 의미로 올바른 것은?

- a) 관계가 단방향이다.
- b) 중개 테이블을 비활성화한다.
- c) 서로 참조 시 자동으로 양방향 관계가 된다.
- d) related_name을 무시한다.

6 related_name은 어떤 기능을 하나요?

- a) 중복 데이터를 제거한다.
- b) 외래키의 on_delete 동작을 설정한다.
- c) 역참조 시 사용할 이름을 지정한다.
- d) 중개 테이블 이름을 설정한다.

8 아래 지문을 읽고 정답을 작성하시오.

Django에서 기본 ManyToManyField를 사용하면 생성되는 중개 테이블의 이름 규칙은?
(‘앱이름’, ‘모델명’, ‘필드명’ 으로 작성)

Q 각 문제에 올바른 답안을 생각해봅시다

9 아래 지문을 읽고 정답을 작성하시오.

다대다(M:N) 관계가 적절하게 사용되는 예시를 하나 이상 작성하기

11 다음 중 symmetrical 옵션을 사용할 수 있는 경우는?

- a) 서로 다른 모델 간 관계
- b) ForeignKey 관계
- c) 자기 자신을 참조하는 다대다 관계
- d) 중개 모델이 있는 관계

10 아래 지문을 읽고 정답을 작성하시오.

add() 메서드로 중개 모델에 값을 저장할 때, 추가 필드 값을 전달하기 위해 사용하는 인자는?

12 중개 테이블을 직접 정의할 때 얻을 수 있는 가장 큰 이점은?

- a) 테이블 이름을 자동으로 설정할 수 있음
- b) 관계 수를 줄일 수 있음
- c) 중복된 관계를 제거할 수 있음
- d) 추가적인 정보를 함께 저장할 수 있음

Q 각 문제에 올바른 답안을 생각해봅시다

13 직접 정의한 중개 모델에서 예약 정보를 삭제할 때 사용하는 메서드?

- a) remove()
- b) clear()
- c) delete()
- d) discard()

14 다음 문장이 올바른지 O, X로 답하십시오.

ManyToOneField는 두 모델 중 어느 쪽에 정의해도 관계에는 영향을 주지 않는다.

15 다음 문장이 올바른지 O, X로 답하십시오.

중개 모델을 정의할 때, 관계를 맺는 두 모델은 모두 ForeignKey로 연결해야 한다.

16 아래 지문을 읽고 정답을 작성하십시오.

중개 모델에서 의사(Doctor)와 환자(Patient)를 연결하기 위해 사용하는 필드는?

확인 문제



정답 및 해설

1. a) 데이터 중복 발생 2. c) ManyToManyField 3. c) through 4. O 5. c) .remove() 6. c) 역참조 시 사용할 이름을 지정한다.
7. c) 서로 참조 시 자동으로 양방향 관계가 된다 8. 앱이름_모델명_필드명

1. N:1 관계에서는 동일한 객체(예: 환자)가 여러 상대(예: 의사)와 연결될 경우, 동일한 데이터가 여러 번 저장되어 중복 문제가 발생할 수 있습니다.
2. Django에서 다대다 관계를 구성할 때는 ManyToManyField를 사용합니다. 이 필드는 두 모델 간의 중간 테이블을 자동으로 생성해 관계를 관리합니다.
3. 중간 테이블에 예약일, 증상 등 추가 정보를 저장하려면, ManyToManyField에 'through' 속성을 설정하여 사용자 정의 중간 모델을 연결해야 합니다.
4. 중간 모델은 단순히 객체 간의 연결뿐 아니라 예약 시간, 증상 등과 같은 부가 정보를 저장할 수 있어 복잡한 관계를 표현하는 데 유용합니다.

5. .remove() 메서드는 중간 테이블에서 특정 관계만 제거하며, 연결된 객체 자체는 삭제하지 않습니다.
6. related_name은 역참조할 때 사용할 매니저 이름을 직접 설정할 수 있게 하여, 이름 충돌이나 가독성 문제를 해결하는 데 사용됩니다.
7. symmetrical=True는 자기 자신을 참조하는 다대다 관계에서 한쪽이 참조하면 다른 쪽도 자동으로 참조하게 만드는 설정입니다. 기본값은 True입니다.
8. Django는 ManyToManyField를 기반으로 자동 생성되는 중간 테이블의 이름을 '앱이름_모델명_필드명' 형식으로 지정합니다.

확인 문제



정답 및 해설

9. 영화와 배우 / 사용자와 좋아요 / 환자와 의사 10. through_defaults 11. c) 자기 자신을 참조하는 다대다 관계
12. d) 추가적인 정보를 함께 저장할 수 있음 13. c) delete() 14. O 15. O 16. ForeignKey

9. 다대다 관계는 한 객체가 여러 객체와 연결되고, 그 반대도 가능한 상황에 사용됩니다. 예: 한 영화에 여러 배우, 한 배우가 여러 영화에 출연.
10. 직접 정의한 중개 모델에 .add() 메서드를 사용할 때, 추가 필드의 값을 전달하려면 through_defaults 인자를 사용합니다.
11. symmetrical 옵션은 ManyToManyField가 'self'를 참조할 때만 사용할 수 있으며, 이 경우 관계의 대칭 여부를 설정할 수 있습니다.
12. 중개 테이블을 직접 정의하면 예약 시간, 증상 등의 부가 정보를 함께 저장할 수 있어, 현실 세계의 복잡한 관계를 정교하게 표현할 수 있습니다.

13. 직접 정의한 중개 모델에서는 .remove()를 사용할 수 없기 때문에, 해당 중개 모델 인스턴스를 찾아 delete() 메서드로 삭제합니다.
14. ManyToManyField는 양방향 관계이므로, 어느 모델에 정의해도 기능은 동일하게 작동합니다. 단, 필드 위치에 따라 참조/역참조 방향이 결정되므로 주의가 필요합니다.
15. 중개 모델은 두 모델 간의 다대다 관계를 표현하기 위해, 각각의 모델을 ForeignKey로 연결하는 방식으로 정의됩니다.
16. 중개 테이블에서는 각각의 모델(예: Doctor, Patient)을 ForeignKey로 연결하여 다대다 관계를 표현합니다.

핵심 키워드

개념	설명	예시
N:M 관계	여러 개의 인스턴스가 서로를 참조하는 관계	학생 ↔ 수업
중개 모델	N:M 관계를 표현하면서 추가 정보를 포함시키는 중간 테이블	예약 ↔ 예약날짜, 예약시간
ManyToManyField	Django에서 N:M 관계를 정의하는 필드	<code>user.likes = models.ManyToManyField(Post)</code>
through	ManyToMany 관계에 커스텀 모델을 지정할 때 사용하는 인자	<code>through='Like'</code>
related_name	역참조 시 사용할 이름 지정	<code>related_name='liked_posts'</code>
symmetrical	관계의 방향성을 설정하는 속성	친구 관계는 대칭적, 팔로우는 비대칭
역참조 충돌	동일 모델 간 다중 관계 발생 시 참조 이름이 겹치는 문제	Like 모델에서 <code>user ↔ post</code>

〈표4_DB_Many To Many Relationships 01_핵심 키워드〉

활동 정리

오늘 공부한 내용 요약 및 정리

- 다대다 관계란?
 - 여러 인스턴스가 서로를 참조하는 복잡한 관계를 표현할 수 있음
 - 환자 ↔ 의사, 유저 ↔ 게시글 같은 상황에 적합함
- 중개 모델의 필요성
 - 단순 연결 외에 추가 정보(예: 예약 시간)를 저장하고 싶을 때 필요
 - 'through' 옵션으로 원하는 중간 모델을 직접 정의할 수 있음
- ManyToManyField 구조 이해
 - Django에서 다대다 관계를 선언하는 필드
 - 기본 중개 테이블을 자동으로 생성하지만 커스터마이징도 가능함
- 중개 모델 설계 방식
 - 예약 시스템에서는 예약 테이블이 중개 역할
 - 예약 시간, 상태 등의 필드를 포함시켜 정보 확장 가능

활동 정리

오늘 공부한 내용 요약 및 정리

- 좋아요 기능 구현 시 고려사항
 - 유저 ↔ 게시글 간 좋아요 관계는 비대칭적 구조
 - 역참조 이름 충돌을 방지하려면 related_name 설정 필요
- symmetrical 속성 활용
 - 관계 방향성을 설정할 수 있음
 - 친구 관계처럼 대칭 구조에는 True, 팔로우처럼 단방향에는 False 설정
- 중복 참조 이슈 해결
 - 같은 모델 간 다중 관계가 있을 경우 명확한 속성 명 정의가 중요
 - 설계 시 혼동을 줄이기 위한 명명 규칙이 필요함

활동 정리

N:1 관계보다 복잡한 관계를 유연하게 설계하는 방안을 고민해봅시다.

- 환자와 의사 간 예약 시스템을 만들 때, N:1 구조만으로는 한계가 있었습니다.
- 중개 테이블을 활용하여 N:1의 한계를 보완할 수 있을 뿐더러 효율적으로 관리할 수 있었습니다.

1. Django의 **중개 모델**을 사용하면 예약 정보와 부가 정보를 함께 저장할 수 있습니다.
2. **ManyToManyField**를 사용하면 양방향 다대다 관계를 쉽게 구성할 수 있습니다.
3. **'through'** 인자를 활용하면 중간 테이블에 사용자 지정 필드를 추가할 수 있습니다.
4. **related_name, symmetrical** 같은 옵션으로 참조 방향성과 충돌을 제어할 수 있습니다.

- 다음 차시에서는 이러한 모델 구조를 기반으로 실제 QuerySet을 활용한 조회 및 수정 방법을 학습할 예정입니다.
- 지금까지의 학습을 바탕으로 실무에 적용 가능한 Django Query를 개선하는 방법 등을 학습할 예정입니다.

←

감사합니다

→

